

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1104

1. Suppose a software system is developed for a Smart Banking System where customers can

open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- **Class**
- **Object**
- **Message Passing**
- **Encapsulation**
- **Polymorphism**

(b) Describe the following object relationships:

- **Association**
- **Aggregation**
- **Composition**

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

1(a) Object-Oriented Concepts in a Smart Banking System

1. Class

A **class** is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) of an object.

Example:

- **Class:** `Customer`
- **Attributes:** `CustomerID`, `Name`, `Address`
- **Methods:** `openAccount()`, `updateDetails()`

Similarly, classes such as `BankAccount`, `Transaction`, and `UtilityBill` can be created.

2. Object

An **object** is an instance of a class. It represents a real-world entity with its own data and behavior.

Example:

- Customer object: `Pooja`
- Account object: `Savings Account No. 10024567`

Each customer or bank account in the system is an object.

3. Message Passing

Message passing is the process by which objects communicate with one another by calling methods.

Example:

When a customer transfers money:

- The `Customer` object sends a message to the `BankAccount` object by calling `transferFunds()`.
- The account object processes the request and updates the balance.

This enables interaction between different objects.

4. Encapsulation

Encapsulation is the process of combining data and methods into a single unit while restricting direct access to data.

Example:

The `BankAccount` class keeps the account balance private.

Customers cannot directly modify the balance; they must use methods such as:

- `deposit()`
- `withdraw()`
- `transferFunds()`

This protects sensitive banking information.

5. Polymorphism

Polymorphism means "one interface, many forms." Different objects can respond differently to the same method.

Example:

Method: `calculateInterest()`

- `SavingsAccount` → Calculates interest at 4%.
- `FixedDepositAccount` → Calculates interest at 7%.

- CurrentAccount → May return no interest.

The same method behaves differently depending on the account type.

1(b) Object Relationships

1. Association

Association represents a relationship where two objects are connected and can interact independently.

Example:

A **Customer** owns one or more **Bank Accounts**.

- A customer can exist without a specific account.
- An account belongs to a customer.

Relationship:

Customer ↔ BankAccount

2. Aggregation

Aggregation is a "has-a" relationship where one object contains another, but the contained object can exist independently.

Example:

A **Bank** has many **Customers**.

Even if the bank branch closes, customer records may still exist in another branch or database.

Relationship:

Bank ◇— Customer

3. Composition

Composition is a strong "part-of" relationship where the child object cannot exist without the parent object.

Example:

A **BankAccount** contains **Transactions**.

If the bank account is permanently deleted, its associated transactions are also removed.

Relationship:

BankAccount ◆— Transaction.

1(c) How These Concepts Improve Software Reusability and Maintainability

1. Reusability

- Classes such as `Customer`, `Account`, and `Transaction` can be reused in different banking modules.
- Inheritance and polymorphism allow new account types (Savings, Current, Fixed Deposit) to reuse existing code.
- Developers avoid rewriting similar code, saving development time.

2. Maintainability

- Encapsulation protects important data such as account balances and customer information.
- Changes made to one class (e.g., modifying the interest calculation) do not affect unrelated classes.
- Message passing keeps communication between objects organized and reduces code dependency.
- Well-defined relationships (association, aggregation, composition) make the system easier to understand, debug, and extend.

3. Flexibility

- New services such as loan management, credit card processing, or online investments can be added without changing the existing banking system significantly.
- Polymorphism allows new account types or payment methods to be integrated with minimal code changes.

4. Security

- Encapsulation ensures that sensitive banking data cannot be modified directly.
- Only authorized methods can access or update account information, improving system security.

Conclusion

Object-oriented concepts such as **classes, objects, message passing, encapsulation, and polymorphism**, along with relationships like **association, aggregation, and composition**, help build a Smart Banking System that is **modular, secure, reusable, scalable, and easy to maintain**. These principles reduce code duplication, improve data protection, and simplify future enhancements.

2. Consider developing a Smart Healthcare Management System.

(a) Explain the following object-oriented design principles:

- **Abstraction**
- **Encapsulation**
- **Modularity**

(b) Describe the following SOLID principles:

- **Liskov Substitution Principle (LSP)**
- **Interface Segregation Principle (ISP)**

(c) Explain how these principles improve software quality and flexibility.

(a) Object-Oriented Design Principles

1. Abstraction

- Hides unnecessary details and shows only essential features.
- Example: A patient books an appointment without knowing how it is stored in the database.

2. Encapsulation

- Combines data and methods into one unit and protects data from direct access.
- Example: Patient medical records are private and can only be accessed through methods like `viewRecord()`.

3. Modularity

- Divides the system into smaller, independent modules.
- Example: Separate modules for Patients, Doctors, Appointments, and Billing.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

- A subclass should be able to replace its parent class without changing the program's behavior.
- Example: `Cardiologist` and `GeneralPhysician` can both be used wherever a `Doctor` object is required.

2. Interface Segregation Principle (ISP)

- A class should not be forced to implement methods it does not use.
- Example: A `Receptionist` uses only the `Billing` interface, while a `Doctor` uses `Diagnosis` and `Prescription` interfaces.

(c) How These Principles Improve Software Quality

- Abstraction reduces complexity.
- Encapsulation protects patient data.
- Modularity makes the system easier to develop and maintain.
- LSP improves code reuse and reliability.
- ISP reduces unnecessary code and increases flexibility.

Conclusion: These principles make the healthcare system secure, reusable, flexible, and easy to maintain.

3. Suppose a software company is developing an Online Food Delivery System.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- **Planning**
- **Requirement Analysis**
- **Design**
- **Development**
- **Testing**
- **Maintenance**

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

(a) Phases of the Software Development Life Cycle (SDLC)

1. Planning

- Defines project goals, budget, timeline, and resources.
- **Example:** Plan features like food ordering and online payment.

2. Requirement Analysis

- Collects and analyzes user requirements.
- **Example:** Customers can order food, track orders, and make payments.

3. Design

- Creates the system architecture and interface.
- **Example:** Design the app screens, database, and order process.

4. Development

- Developers write the program code.
- **Example:** Build customer, restaurant, and delivery modules.

5. Testing

- Checks the software for bugs and errors.
- **Example:** Test login, ordering, payment, and tracking features.

6. Maintenance

- Updates and fixes the software after release.
- **Example:** Add new features and fix reported issues.

(b) Purpose of Each SDLC Phase

- **Planning:** Defines project scope and objectives.
- **Requirement Analysis:** Understands user needs.
- **Design:** Creates a clear system structure.
- **Development:** Converts the design into working software.
- **Testing:** Ensures the software works correctly.
- **Maintenance:** Keeps the software updated and reliable.

(c) How SDLC Contributes to Reliable Software

- Ensures clear planning and requirements.
- Reduces errors through proper design and testing.
- Improves software quality and performance.
- Makes maintenance and future updates easier.
- Helps deliver software on time and within budget.

Conclusion: SDLC provides a structured process for developing an **Online Food Delivery System**, resulting in software that is **reliable, efficient, and easy to maintain**.

4. Consider a project for developing a Smart Agriculture Monitoring System, where customer requirements change regularly.

(a) Explain the following SDLC models:

- **Prototype Model**
- **Spiral Model**
- **Agile Model**
- **DevOps Model**

(b) Compare the Prototype Model and Agile Model.

(c) Recommend the most suitable SDLC model for this project with justification.

(a) SDLC Models

- **Prototype Model:** Builds a sample system first to collect customer feedback.
- **Spiral Model:** Develops the software in cycles with risk analysis.
- **Agile Model:** Develops software in small iterations with regular customer feedback.
- **DevOps Model:** Supports continuous development, testing, and deployment.

(b) Prototype Model vs Agile Model

Prototype Model	Agile Model
Builds a prototype first.	Develops in small iterations.
Used for unclear requirements.	Used for changing requirements.
Feedback is taken on the prototype.	Feedback is taken after each iteration.

(c) Best SDLC Model

Agile Model

Reason: Customer requirements change regularly. Agile allows quick changes, frequent feedback, and continuous improvement, making it the most suitable model.

5. Suppose a software analyst is designing a Smart Transport Management System.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- **State Diagram**
- **Component Diagram**
- **Deployment Diagram**
- **Package Diagram**

(c) Explain how UML models improve communication among stakeholders.

(a) Importance of UML

- UML is used to design and visualize software.
- It helps developers understand the system before coding.

(b) UML Diagrams

- **State Diagram:** Shows the states of an object.
- **Component Diagram:** Shows software components and their connections.
- **Deployment Diagram:** Shows hardware and software deployment.
- **Package Diagram:** Groups related classes into packages.

(c) How UML Improves Communication

- Makes the system easy to understand.
- Improves communication among developers and customers.
- Reduces errors and simplifies maintenance.

6. Consider developing a Smart Inventory Management System.

(a) Explain the concept of modular software design.

(b) Compare the procedural programming approach with the object-oriented programming

approach.

(c) Explain how modularity improves software scalability, maintainability, and reusability.

(a) Modular Software Design

- **Modular design** divides a system into small, independent modules.
- **Example:** Inventory, Sales, Orders, and Reports modules.

(b) Procedural vs Object-Oriented Programming

Procedural	Object-Oriented
Focuses on functions.	Focuses on objects and classes.
Data and functions are separate.	Data and methods are together.
Less reusable.	More reusable and secure.

(c) Benefits of Modularity

- **Scalability:** New modules can be added easily.
- **Maintainability:** Changes in one module do not affect others.
- **Reusability:** Modules can be reused in other applications.

Conclusion: Modularity makes the Smart Inventory Management System **easy to develop, maintain, and expand.**